

异常处理机制

什么是异常？

- **异常**：程序在运行期间遇到的、使正常执行流程无法继续的事件
- 观察下面这段小程序：

```
num = int(input("请输入一个整数: "))
print(num ** 2)
```

- 当用户的输入不是数字：

```
请输入一个整数: NO
Traceback (most recent call last):
  File "D:/test.py", line 1, in <module>
    num = int(input("请输入一个整数: "))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'NO'
```

语法错误 vs 异常

- **语法错误**：代码没写对，程序一行都不会执行

```
print("A")
while True:
print("B")
```

报错：

```
  File "/Users/hai/Desktop/test.py", line 3
    print("B")
    ^^^^^
IndentationError: expected an indented block after 'while' statement on line
2
```

⚠ 第 1 行的 print("A") **没有输出**。Python 在解析阶段就拒绝了整个文件

语法错误 vs 异常

- **异常**：代码写对了，但运行时出了状况

```
print("A")
print(10 / 0)
```

报错：

```
Traceback (most recent call last):
  File "D:/test.py", line 2, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

⚠ 第 1 行**正常输出** A，程序确实跑起来了，执行到第 2 行才中断

读懂报错信息：Traceback

- **Traceback（回溯）**： Python 留下的"事故报告"，它有固定结构：

```
请输入一个整数: NO
Traceback (most recent call last):
  File "D:/test.py", line 1, in <module>
    num = int(input("请输入一个整数: "))
              ^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'NO'
```

回溯标记

文件 / 行号 / 所在函数

出错的源代码

精确定位

异常类型

异常信息

⚠ Traceback 里最重要的是**异常类型**（ValueError、ZeroDivisionError……）。
它既说明了出错原因，也是下一步程序处理该异常的唯一依据

读懂报错信息：Traceback

- **Traceback（回溯）**阅读顺序：从下往上读

顺序	看哪里	得到什么
第 1 步	**最后一行**	出了什么错（`ValueError`）
第 2 步	倒数第二组 `File`	错在哪个文件、第几行
第 3 步	往上翻	这行代码是被谁调用的

读懂报错信息：Traceback

- 当涉及函数调用时，Traceback 会有多层：

```
def get_num(prompt):
    return int(input(prompt))

def square(prompt):
    n = get_num(prompt)
    return n ** 2

print(square("请输入一个整数: "))
```

读懂报错信息：Traceback

- 当涉及函数调用时，Traceback 会有多层：

```
请输入一个整数: NO
Traceback (most recent call last):
  File "D:/test.py", line 8, in <module>          ← 最外层：程序从这里出发
    print(square("请输入一个整数: "))
  File "D:/test.py", line 5, in square            ← square 调用了 get_num
    n = get_num(prompt)
  File "D:/test.py", line 2, in get_num           ← 真正出错的地方
    return int(input(prompt))
           ^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'NO'
```

⚠ 调用顺序从上往下，最后一个才是真正出事的现场

异常处理: try-except语句

- 语法格式：

```
try:
    <可能出错的语句块>
except <异常类型>:
    <出错后的处理语句块>
```

- 对程序进行改造：

```
try:
    num = int(input("请输入一个整数: "))
    print(num ** 2)
except ValueError:
    print("输入错误，请输入一个整数")

print("程序继续执行")
```

正常输入：

```
请输入一个整数：12
144
程序继续执行
```

异常输入：

```
请输入一个整数：NO
输入错误，请输入一个整数
程序继续执行
```

异常处理: try-except语句

- 语法格式：

```
try:
    <可能出错的语句块>
except <异常类型>:
    <出错后的处理语句块>
```

- 执行规则：

1. 先执行 `try` 子句；
2. **没出异常** → 跳过 `except`，正常往下走；
3. **出了异常，且类型匹配** → `try` 中剩余语句被跳过，转去执行 `except` 子句，然后从整个 try 语句**之后**继续；
4. **出了异常，但类型不匹配** → 不处理，照常崩溃退出。

异常处理: try-except语句

- **验证规则 3**：try 中剩余语句被跳过

```
try:
    print("① 开始")
    num = int("NO")
    print("② 转换成功")
except ValueError:
    print("③ 输入错误，请输入一个整数")
print("④ 程序继续执行")
```

运行结果：
① 开始
③ 输入错误，请输入一个整数
④ 程序继续执行

⚠ ② 没有输出——异常一旦发生，try 块中后面的语句全部作废，**直接跳到 except**

异常处理: try-except语句

- **验证规则 4**：类型不匹配无法处理异常

```
try:
    print(10 / 0)
except ValueError:                # 类型写错了
    print("这里接不住")
```

运行结果：

```
Traceback (most recent call last):
  File “D:/test.py”, line 2, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

⚠ 抛的是 ZeroDivisionError，而 except 写的是 ValueError，无法匹配

异常处理: try-except语句

- 获取异常信息：except ... as e：
  - 在异常类型后加 as 变量名，就能得到异常对象本身：

```
try:
    num = int("NO")
except ValueError as e:
    print("异常类型: ", type(e).__name__)
    print("异常信息: ", e)
    print("异常参数: ", e.args)
```

```
异常类型:  ValueError
异常信息:  invalid literal for int() with base 10: 'NO'
异常参数:  ("invalid literal for int() with base 10: 'NO'",)
```

⚠ e 只是习惯写法（error / exception 的首字母），写成 err、ex 都可以

多个 except 子句与异常的继承层次

- 多分支异常处理与异常继承层次
  - 一段代码可能引发多种类型的异常。try 语句允许附带多个 except 子句，分别处理不同类型：

```
try:
    <语句块>
except <异常类型 1>:
    <处理语句块 1>
except <异常类型 2>:
    <处理语句块 2>
```

⚠ 一个 try 语句可以包含多个 except 子句，但至多只有一个会被执行。异常一旦被某个子句匹配并处理，其余子句均被跳过

多个 except 子句与异常的继承层次

- 多分支异常处理与异常继承层次
  - **示例**：整数转换与除法运算可能引发两类不同的异常

```
try:
    print(int(x) / int(y))
except ValueError:
    print("输入的内容无法转换为整数")
except ZeroDivisionError:
    print("除数不能为 0")
```

多个 except 子句与异常的继承层次

- 多分支异常处理与异常继承层次
  - 若多种异常的处理方式相同，可用元组在一个子句中同时指定：

```
for value in ["10", "NO", None]:
    try:
        print(int(value) / 2)
    except (ValueError, TypeError) as e:
        print(type(e).__name__, ": ", e)
```

运行结果：
5.0
ValueError ： invalid literal for int() with base 10: 'NO'
TypeError ： int() argument must be a string, a bytes-like object or a real number, not 'NoneType'

多个 except 子句与异常的继承层次

- 异常的继承层次
  - Python 的所有内置异常构成一棵继承树
  - 常用部分如下：

```
BaseException                    所有异常的基类
├── SystemExit                   sys.exit() 引发
├── KeyboardInterrupt            用户按下 Ctrl+C 引发
└── Exception                    常规异常的基类
    ├── ArithmeticError
    │   └── ZeroDivisionError
    ├── LookupError
    │   ├── IndexError
    │   └── KeyError
    ├── OSError
    │   └── FileNotFoundError
    ├── ValueError
    ├── TypeError
    ├── NameError
    └── AttributeError
```

多个 except 子句与异常的继承层次

- 异常的继承层次
  - except 子句中指定的类，可以匹配该类本身及其所有派生类的实例；反之不成立，指定派生类的子句无法匹配其基类的实例
  - 由此引出一条必须遵守的规则：派生类应写在基类之前
    - 多分支的匹配过程自上而下逐一进行，首个匹配成功的子句被执行
    - 先写具体的派生类，再写笼统的基类，最后才是兜底的 Exception

多个 except 子句与异常的继承层次

- 异常的继承层次
  - 顺序错误：基类在前

```
try:
    num = int("NO")
except Exception:
    print("发生了未知错误")
except ValueError:
    print("输入的内容无法转换为整数")
```

运行结果：
发生了未知错误

⚠ ValueError 首先被 except Exception 匹配，第二个子句永远不会被执行

多个 except 子句与异常的继承层次

- 异常的继承层次
  - 顺序正确：派生类在前

```
try:
    num = int("NO")
except ValueError:
    print("输入的内容无法转换为整数")
except Exception:
    print("发生了未知错误")
```

运行结果：
输入的内容无法转换为整数



多个 except 子句与异常的继承层次

- Exception 与 BaseException 的区别
  - **Exception** 是绝大多数常规异常的基类，except Exception: 可作为兜底分支使用，但必须置于所有其他分支之后
  - **BaseException** 之下还有两个不属于 Exception 的类：SystemExit 与 KeyboardInterrupt，二者表示程序应当终止

多个 except 子句与异常的继承层次

- Exception 作为兜底分支

```
for value in ["NO", "0"]:
    try:
        print(10 / int(value))
    except ValueError:
        print("输入的内容无法转换为整数")
    except Exception as e:
        print("发生了未知错误: ", type(e).__name__, "-", e)
```

运行结果：
输入的内容无法转换为整数
发生了未知错误： ZeroDivisionError - division by zero

⚠ 已预见的 ValueError 由专门的子句处理；未曾预见的 ZeroDivisionError 则落入兜底分支，程序得以继续运行

else 子句与 finally 子句

- try 语句的完整结构：else 与 finally
  - try 语句除 except 外，还可附加 else 与 finally 两个子句，构成完整结构：

```
try:
    <可能出错的语句块>
except <异常类型>:
    <出错时执行的语句块>
else:
    <未出错时执行的语句块>
finally:
    <无论是否出错都执行的语句块>
```

- try: 总是首先执行
- except: 仅当发生匹配的异常时执行
- else: 仅当 try 块未发生任何异常时执行
- finally: 无论如何都执行，且总是最后执行

else 子句与 finally 子句

- try 语句的完整结构：else 与 finally
  - try 语句除 except 外，还可附加 else 与 finally 两个子句，构成完整结构：

```
try:
    num = int(input("请输入一个整数: "))
except ValueError:
    print("输入错误，请输入一个整数")
else:
    print("它的平方是", num ** 2)
finally:
    print("本次处理结束")
```

两次运行的结果:

请输入一个整数：12
它的平方是 144
本次处理结束

请输入一个整数：NO
输入错误，请输入一个整数
本次处理结束

else 子句与 finally 子句

- try 语句的完整结构：else 与 finally
  - else 作用：缩小 try 块的保护范围
    - try 块中只放真正可能引发目标异常的那一行或几行，其余后续处理一律放入 else
  - finally 的两条规则：
    - **规则一**：无论是否发生异常，finally 都会执行
    - **规则二**：若异常未被任何 except 子句处理，它将在 finally 执行完毕后被重新抛出

else 子句与 finally 子句

- try 语句的完整结构：else 与 finally
  - **规则二**：若异常未被任何 except 子句处理，它将在 finally 执行完毕后被重新抛出

```
try:
    print(10 / 0)
except ValueError:                  # 类型不匹配，无法捕获
    print("这里无法捕获")
finally:
    print("finally 子句仍然执行")
```

```
finally 子句仍然执行
Traceback (most recent call last):
  File "D:/test.py", line 2, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

⚠ finally 的内容**先于**报错信息出现

else 子句与 finally 子句

- try 语句的完整结构：else 与 finally
  - finally 的典型用途：执行必须完成的收尾工作——关闭文件、释放资源、断开网络连接、记录运行日志等
    - 这类操作不能依赖程序"正常走完"，因为一旦中途出错就会被跳过；写入 finally 则可确保其在任何情况下都被执行

```
f = open("data.txt")
print(f.read())
print(10 / 0)                    # 此处发生异常
f.close()                        # 永远不会执行
```

⚠ 文件使用完毕后必须关闭。若不作任何保护，一旦中途出错，关闭语句将被跳过

else 子句与 finally 子句

- try 语句的完整结构：else 与 finally
  - finally 的典型用途：执行必须完成的收尾工作

```
f = open("data.txt")
try:
    print(f.read())
    print(10 / 0)
finally:
    f.close()
    print("文件已关闭，当前状态 closed =", f.closed)
```

```
文件内容第一行
文件已关闭，当前状态 closed = True
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
```

⚠ 异常照常向外抛出，但关闭操作已经完成

主动抛出异常——raise 语句

- 为何需要主动抛出异常？
  - 因为Python 只能检测语言层面的错误，无法判断业务层面的合法性

```
score = int(input("请输入成绩: "))
print("成绩为: ", score)
```

⚠ 120 是一个完全合法的整数，int() 转换毫无问题，Python 不会给出任何提示。但在成绩录入这一业务场景中，该数据显然是无效的 (百分制)

主动抛出异常——raise 语句

- raise 语法格式
  - raise <异常类型>("说明信息")

```
score = int(input("请输入成绩: "))
if score < 0 or score > 100:
    raise ValueError("成绩必须在 0 到 100 之间")
print("成绩为: ", score)
```

```
请输入成绩: 120
Traceback (most recent call last):
  File "D:/test.py", line 3, in <module>
    raise ValueError("成绩必须在 0 到 100 之间")
ValueError: 成绩必须在 0 到 100 之间
```

⚠ 程序按照与内置异常完全一致的方式终止并报错。
raise 之后的语句不再执行

主动抛出异常——raise 语句

- raise 与 try-except 配合使用
  - 主动抛出的异常，同样可以被 except 子句捕获：

```
for v in ["85", "120", "NO"]:
    try:
        score = int(v)
        if score < 0 or score > 100:
            raise ValueError("成绩必须在 0 到 100 之间")
    except ValueError as e:
        print("输入有误: ", e)
    else:
        print("成绩为: ", score)
```

`"85"` —— 合法数据，进入 `else` 分支；

`"120"` —— 由 **`raise`** 主动抛出的 `ValueError`；

`"NO"` —— 由 **`int()`** 自动抛出的 `ValueError`。

⚠ 后两者虽然来源不同，却**属于同一类型**，因而被同一个 except 子句统一处理

主动抛出异常——raise 语句

- raise 的合适异常类型
  - 主动抛出时应优先选用语义匹配的内置异常，而非一律使用 Exception：

场景	推荐类型
值超出允许范围、格式不合法	`ValueError`
参数类型不正确	`TypeError`
索引或键不存在	`IndexError` / `KeyError`

主动抛出异常——raise 语句

- 不带参数的 raise：重新抛出当前异常
  - 在 except 子句内部，单独使用 raise 可将刚捕获的异常原样继续抛出：

```
try:
    num = int("NO")
except ValueError as e:
    print("日志: 程序发生异常 -", e)
    raise
```

```
日志: 程序发生异常 - invalid literal for int() with base 10: 'NO'
Traceback (most recent call last):
  File "D:/test.py", line 2, in <module>
    num = int("NO")
          ^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'NO'
```

异常处理的最佳实践

- 请先审视以下代码，指出其中存在的问题：

```
scores = ["88", "92", "x", "75"]
total = 0

for s in scores:
    try:
        total += int(s)
    except:
        pass

print("总分: ", total, " 平均分: ", total / len(scores))
```

运行结果：总分： 255 平均分： 63.75

三名学生的实际平均分为 85 分，程序却给出 63.75 分

异常处理的最佳实践

- 请先审视以下代码，指出其中存在的问题：

```
scores = ["88", "92", "x", "75"]
total = 0

for s in scores:
    try:
        total += int(s)
    except:
        pass

print("总分: ", total, " 平均分: ", total / len(scores))
```

误用一：使用不带类型的 except:
等价于 except BaseException:，将捕获一切异常，包括表示程序应当终止的 KeyboardInterrupt 与 SystemExit

误用二：except 之后接 pass:
发生了异常，但不做任何记录也不做任何处理

异常处理的最佳实践

- 改进后的完整写法：

```
scores = ["88", "92", "x", "75"]
total = 0
count = 0

for s in scores:
    try:
        total += int(s)
        count += 1
    except ValueError:
        print("警告：数据", s, "无法转换，已跳过")

print("总分：", total, " 有效数据：", count, " 平均分：", total / count)
```

运行结果：
警告：数据 x 无法转换，已跳过 总分： 255 有效数据： 3 平均分： 85.0

常见异常类型速查

⚠ 前两类属于**语法错误**，在程序启动前即被检出，**无法捕获**，只能修改代码；其余均为**异常**，可由 try-except 处理

异常类型	触发原因	示例代码	报错信息
`SyntaxError`	语法不符合规则	`int int`	`invalid syntax`
`IndentationError`	缩进不正确	该缩进处未缩进	`expected an indented block after ...`
`NameError`	使用了未定义的变量	`print(v)`	`name 'v' is not defined`
`ValueError`	类型正确但值不合法	`int("NO")`	`invalid literal for int() with base 10: 'NO'`
`TypeError`	类型不匹配	`"2" + 2`	`can only concatenate str (not "int") to str`
`ZeroDivisionError`	除数为 0	`1 / 0`	`division by zero`
`IndexError`	序列索引超出范围	`[1,2,3][10]`	`list index out of range`
`KeyError`	字典中不存在该键	`{'a':1}['b']`	`'b'`
`AttributeError`	对象没有该属性或方法	`"abc".push(1)`	`'str' object has no attribute 'push'`
`FileNotFoundError`	文件不存在	`open("nope.txt")`	`[Errno 2] No such file or directory: 'nope.txt'`
`ModuleNotFoundError`	模块不存在	`import nomodule`	`No module named 'nomodule'`

练习题

- **第 1 题** 以下程序的输出是什么？

```
try:
    print("A")
    print(10 / 0)
    print("B")
except ZeroDivisionError:
    print("C")
else:
    print("D")
finally:
    print("E")
print("F")
```

运行结果：

A
C
E
F

练习题

- **第 1 题** 以下程序的输出是什么？

步骤	执行语句	结果
①	`print("A")`	输出 **A**
②	`print(10 / 0)`	引发 `ZeroDivisionError`
③	`print("B")`	**跳过**——异常发生后 `try` 块剩余语句不再执行
④	匹配 `except ZeroDivisionError`	类型一致，输出 **C**
⑤	`else` 子句	**跳过**——`else` 仅在**无异常**时执行
⑥	`finally` 子句	输出 **E**——无条件执行
⑦	`print("F")`	异常已被处理，程序正常继续，输出 **F**

练习题

- **第 2 题** 以下程序的输出是什么？

```
try:
    print("A")
    print(10 / 0)
    print("B")
except ValueError:
    print("C")
else:
    print("D")
finally:
    print("E")
print("F")
```

运行结果：

A
E
Traceback (most recent call last): File "demo.py", line 3, in <module> print(10 / 0) ~~~^~~ ZeroDivisionError: division by zero

练习题

- **第 3 题** 以下程序输出"甲"还是"乙"？为什么？

```
try:
    num = int("NO")
except Exception:
    print("甲")
except ValueError:
    print("乙")
```

输出"甲"。ValueError 是 Exception 的派生类，首先被第一个子句匹配，第二个子句**永远不会执行**。基类必须写在派生类之后

练习题

- **第 4 题** 以下程序的输出顺序是什么？程序会崩溃吗？

```
try:
    print(10 / 0)
except ValueError:
    print("甲")
finally:
    print("乙")
print("丙")
```

先输出"乙"，随后程序崩溃并显示 ZeroDivisionError 的报错信息，"甲"和"丙"均不输出。

因为异常类型不匹配，无法被捕获；但 finally 仍然执行，执行完毕后异常被重新抛出。

编程练习题

已知一批学生成绩数据，其中混有无效数据。编写程序完成筛选与统计。

```python
records = ["88", "92", "x", "75", "120", "60"]
```

要求

1. 遍历 `records`，逐条处理；
2. 有效成绩的标准是：**能转换为整数**，且在 **0 到 100 之间**；
3. 对两类无效数据分别给出提示，**程序不得中断**；
4. 最后输出有效成绩条数、总分、平均分（保留两位小数）。